

```
0000 -----
compiler-- Copyright (C) 2024-2026 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/
or modify-- it under the terms of the GNU General Public License as published by-- the Free Software Foundation, either version 3 of the License, or
-- (at your option) any later version.-- This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without eve
n the implied warranty of-- MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the-- GNU General Public License for more details.-- You sh
ould have received a copy of the GNU General Public License-- along with this program. If not, see <https://www.gnu.org/licenses/>.--
-----
-- WITH SEQUENTIAL_IO; WITH TEXT_IO; SEPARATE(-IDL)
-- IDL_TBL.ADB-- Vincent MORIN
-----
-- 1-2-3-4-5-6-7-8-9-A-B-C
-----
PACKAGE BODY IDL
  TBLS IS
    0010 PROCEDURE INIT_SPEC ( SPEC_FILE : STRING ) IS
      PACKAGE INT_IO IS NEW INTEGER_IO ( INTEGER );
      USE INT_IO;
      USE TEXT_IO;
      SFILE : FILE_TYPE;
      T
      _CHR : CHARACTER;
      T_INT : INTEGER;
      T_TXT : STRING ( 1 .. 50 );
      T_LAST : NATURAL;
      AS_SEEN : BOOLEAN;
      AS_LIST_SEEN : BOOLEAN;
      NON
      AS_SEEN : BOOLEAN;
      BEGIN
        OPEN ( SFILE, IN_FILE, SPEC_FILE & ".tbl" );
        LAST_NODE := -1;
        LAST_ATTR := -1;
        LAST_NODE_ATTR := 0;
        WHILE NOT END_O
        F_FILE ( SFILE ) LOOP
          GET ( SFILE, T_CHR );
          PRENDRE LE CARACTERE INDIQUANT LE TYPE DE LIGNE
          IF T_CHR /= 'C' AND T_CHR /= 'E' THEN
            PAS CLASSE ( C OU E ) DONC 'N' OU 'A'
            GET ( SFILE, T_INT );
            N DE-NOEUD OU D'ATTRIBUT ( NEGATIF )
            END IF;
            GET_LINE ( SFILE, T
            TXT, T_LAST );
            PRENDRE LE RESTANT DE LA LIGNE
            SUPPRESS_BLANKS;
            DECLARE
              D : INTEGER := 0;
              BEGIN
                FOR S IN 1 .. T_LAS
                T LOOP
                  IF T_TXT ( S ) /= ' ' AND T_TXT ( S ) /= ASCII.HT THEN
                    CARACTERE NON ' ' OU TAB
                    D := D + 1;
                    AVANCER LE POINTEUR DE DES
                    TINATION
                    T_TXT ( D ) := T_TXT ( S );
                    RECOPIER LE CARACTERE
                    END IF;
                    END LOOP;
                    T_LAST := D;
                    LONGUEUR REDUITE DE
                    S ESPACES
                    END SUPPRESS_BLANKS;
                    NOEUDS
                    IF T_CHR = 'N' THEN
                      UNE LIGNE DECLARANT UN NOEUD
                      LAST_NODE := LAST_NODE + 1;
                      UN NOEUD DE PLUS
                      IF LAST_NODE /= T_INT THEN
                        LE NUMERO D'ORDRE DOIT CORRESPONDRE-AU NUMERO-D'IDENTIFICATION
                        PUT_LINE (
                        0020 "IDL.IDL_TBL.INIT_SPEC: LAST NODE /= T_INT" );
                        RAISE PROGRAM_ERROR;
                        END IF;
                        N_SPEC ( NODE_NAME'VAL ( LAST_NODE ) ) := ( NS_SIZE =>
                        0,
                        PAS D'ATTRIBUT, -DONC TAILLE NULLE
                        NS_FIRST_A => 0,
                        PAS D'ATTRIBUT ENCORE VU, -NUMERO DU-PREMIER -0
                        NS_ARITY => NULLARY
                        );
                        AS_LIST_SEEN := FALSE;
                        PAS VU D' "as" -LIST
                        AS_SEEN := FALSE;
                        PAS VU D' "as"
                        NON_AS_SEEN := FALSE;
                        PAS VU DE NON "as" ( UN "xd" OU "sm" ... )
                        ATTRIBUTS
                        ELIF T_CHR = 'A' OR T_CHR = 'B' OR T_CHR = 'I' THEN
                          LAST_NODE_ATTR
                          := LAST_NODE_ATTR + 1;
                          UN ATTRIBUT DE PLUS
                          DECLARE
                            NN : NODE_NAME := NODE_NAME'VAL ( LAST_NODE );
                            BEGIN
                              IF N_SPEC
                              ( NN ).NS_FIRST_A = 0 THEN
                                SI L'ON A PAS VU LE PREMIER ATTRIBUT
                                N_SPEC ( NN ).NS_FIRST_A := LAST_NODE_ATTR;
                                METTRE L'INDICE-DE CET AT
                                TRIBUT COMME PREMIER
                                END IF;
                                N_SPEC ( NN ).NS_SIZE := N_SPEC ( NN ).NS_SIZE + 1;
                                INCREMENTER LA TAILLE DU NOEUD AUQUEL ON AJOUT
                                E L'ATTRIBUT
                                IF T_LAST >= 3 AND THEN T_TXT ( 1 .. 3 ) = "as" THEN
                                  ATTRIBUT COMMENANT PAR "as"
                                  IF T_INT < 0 THEN
                                    IDENTIFICA
                                    TEUR NEGATIF ( REPERE UNE LISTE, UN SEQ_TYPE )
                                    IF AS_SEEN OR NON_AS_SEEN THEN
                                      ON A DEJ VU UN-AS OU UN-NON AS, INTERDIT -UNE "AS" -LIST DO
                                      IT-ARRIVER EN TTE
                                      PUT_LINE ( "BAD AS LIST: " & T_TXT ( 1 .. T_LAST ) );
                                      END IF;
                                      AS_SEEN := TRUE;
                                      VU UN-"AS"
                                      AS_LIST_SEEN := TR
                                      UE;
                                      VU UNE "AS" LIST
                                      N_SPEC ( NN ).NS_ARITY := ARITIES'VAL ( ARITIES'POS ( N_SPEC ( NN ).NS_ARITY ) + 4 );
                                      ELSE
                                      IDENTIFICATEUR P
                                      OSITIF ( UN "AS" QUI N'EST PAS UN SEQ_TYPE )
                                      IF AS_LIST_SEEN OR NON_AS_SEEN THEN
                                      ON NE-DOIT PAS AVOIR DE "AS" LIST AVANT UN "AS" NON LIST
                                      E ET PAS DE NON "AS" NON PLUS
                                      PUT_LINE ( "BAD AS ... " & T_TXT ( 1 .. T_LAST ) );
                                      END IF;
                                      AS_SEEN := TRUE;
                                      VU UN-"AS"
                                      N_SPEC (
                                      NN ).NS_ARITY := ARITIES'VAL ( ARITIES'POS ( N_SPEC ( NN ).NS_ARITY ) + 1 );
                                      END IF;
                                      ELSE
                                      PAS UN "AS" -NON_AS_SEEN := TRUE;
                                      NON "AS" VU
                                      END IF;
                                      END;
                                      A_SPEC ( LAST_NODE_ATTR ).IS_LIST := T_INT < 0;
                                      INDICATEUR DE LISTE ( UNE SEULE PAR NOEUD )
                                      T_INT := ABS T_INT;
                                      IDENTIFICATEUR EN POSITIF
                                      A_SPEC ( LAST_NODE_ATTR ).ATTR := ATTRIBUTE_NAME'VAL ( T_INT );
                                      STOCKER L'IDEN
                                      TIFICATEUR DE L'ATTRIBUT
                                      IF T_INT > LAST_ATTR THEN
                                      DEPASSE LE NOMBRE D'ATTRIBUTS VUS
                                      LAST_ATTR := T_INT;
                                      METTRE JO
                                      UR CE NOMBRE ( NOTRE ATTRIBUT INDIQUE QU'IL Y-EN A PLUS )
                                      END IF;
                                      END IF;
                                      END LOOP;
                                      CLOSE ( SFILE );
                                      END INIT_SPEC;
                                      -----
                                      PACKAGE DTT_IO IS NEW SEQUENTIAL_IO ( DIANA_TABLE_TYPE );
                                      USE DTT
                                      IO;
                                      0040 TE_SPEC ( SPEC_FILE : STRING ) IS
                                        SFILE : FILE_TYPE;
                                        BEGIN
                                          DTT_IO.CREATE ( SFILE, OUT_FILE, SPEC_FILE & ".bin" );
                                          DTT_IO.WRITE ( SFILE, DI
                                          ANA_TABLE_AREA );
                                          DTT_IO.CLOSE ( SFILE );
                                          END;
                                          -----
                                          PROCEDURE READ_SPEC
                                          -----
                                          PROCEDURE READ_SPEC ( SPEC_FILE : STRING ) IS
                                            SFILE : FILE_TYPE;
                                            BEGIN
                                              DTT_IO.OPEN ( SFILE, IN_FILE, SPEC_FILE
                                              & ".bin" );
                                              DTT_IO.READ ( SFILE, DIANA_TABLE_AREA );
                                              DTT_IO.CLOSE ( SFILE );
                                              END;
                                              -----
                                              END IDL_TBL;
                                              -----
                                              1-2-3-4-5-6-7-8-9-A-B-C
                                              -----

```